Lab Taks-5

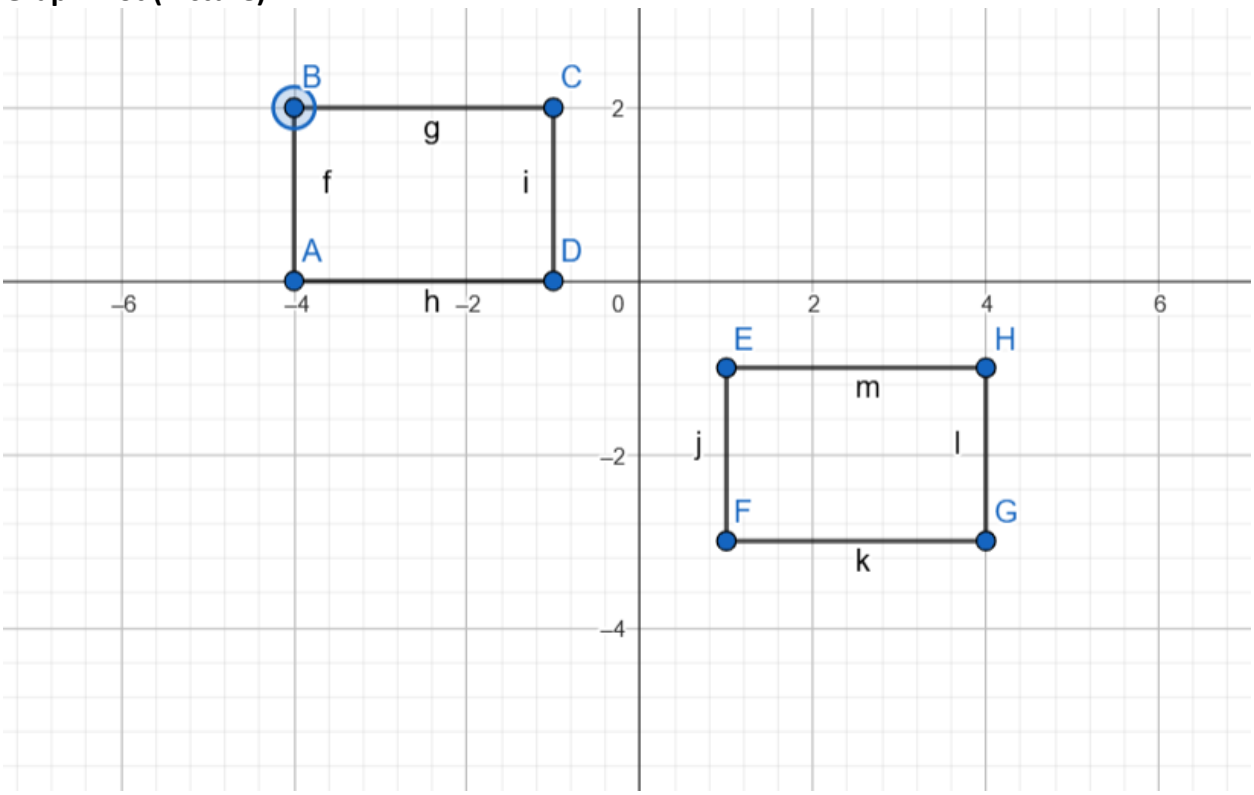
Submission Guidelines-

- Rename the file with your serial number only
- Must submit within the announced time.
- Must include resources for all the section in the table

Question-1

Create an animation using two box that will move in the opposite direction.

Graph Plot (Picture)-



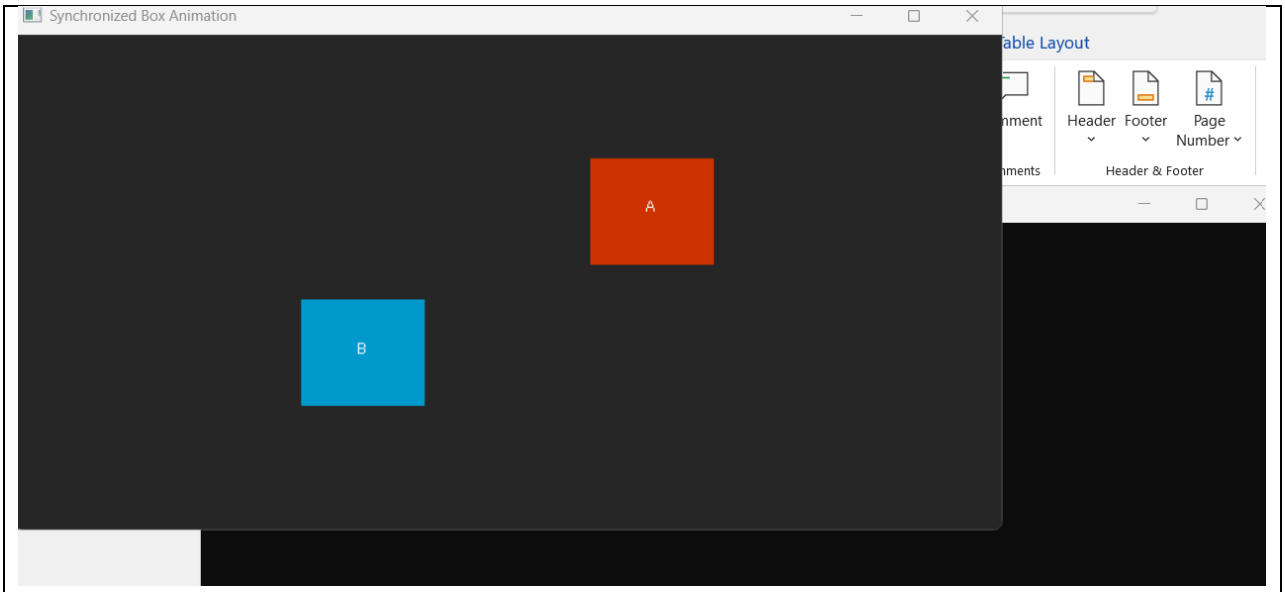
Code-

```
#include <GL/glut.h>

struct Box {
   float x, y;
   float width, height;
   float speed;
   int direction;
   float color[3];
```

```
   const char* label;
};

Box box1 = {-3.0f, 1.0f, 2.0f, 1.5f, 0.03f, 1, {0.8f, 0.2f, 0.0f}, "A"}; // Orange box moving right
Box box2 = {3.0f, -1.0f, 2.0f, 1.5f, 0.03f, -1, {0.0f, 0.6f, 0.8f}, "B"};  // Teal box moving left

void drawText(float x, float y, const char* text) {
   glRasterPos2f(x, y);
   while (*text) {
      glutBitmapCharacter(GLUT_BITMAP_HELVETICA_12, *text++);
   }
}

void drawBox(const Box& box) {
   glColor3fv(box.color);
   glBegin(GL_QUADS);
      glVertex2f(box.x - box.width/2, box.y - box.height/2);
      glVertex2f(box.x + box.width/2, box.y - box.height/2);
      glVertex2f(box.x + box.width/2, box.y + box.height/2);
      glVertex2f(box.x - box.width/2, box.y + box.height/2);
   glEnd();

   // Draw label at box center
   glColor3f(1.0f, 1.0f, 1.0f);
   drawText(box.x - 0.1f, box.y, box.label);
}

void update(int value) {
   // Move both boxes at same speed in opposite directions
   box1.x += box1.speed * box1.direction;
   box2.x += box2.speed * box2.direction;

   // Boundary checking - bounce when edges hit boundaries
   if (box1.x + box1.width/2 > 7.0f || box1.x - box1.width/2 < -7.0f) {
      box1.direction *= -1;
   }
   if (box2.x + box2.width/2 > 7.0f || box2.x - box2.width/2 < -7.0f) {
      box2.direction *= -1;
   }

   glutPostRedisplay();
   glutTimerFunc(16, update, 0);
}
```

```
void display() {
   glClear(GL_COLOR_BUFFER_BIT);

   // Draw title and info
   //glColor3f(1.0f, 1.0f, 1.0f);
   //drawText(-1.5f, 2.5f, "Synchronized Box Movement");
   //drawText(-3.0f, -2.8f, "Both boxes moving at speed: 0.03");

   drawBox(box1);
   drawBox(box2);

   glutSwapBuffers();
}

void init() {
   glClearColor(0.15f, 0.15f, 0.15f, 1.0f); // Dark background
   glMatrixMode(GL_PROJECTION);
   glLoadIdentity();
   gluOrtho2D(-8.0, 8.0, -3.5, 3.5);
}

int main(int argc, char** argv) {
   glutInit(&argc, argv);
   glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
   glutInitWindowSize(800, 400);
   glutCreateWindow("Synchronized Box Animation");

   init();
   glutDisplayFunc(display);
   glutTimerFunc(0, update, 0);
   glutMainLoop();

   return 0;
}
```

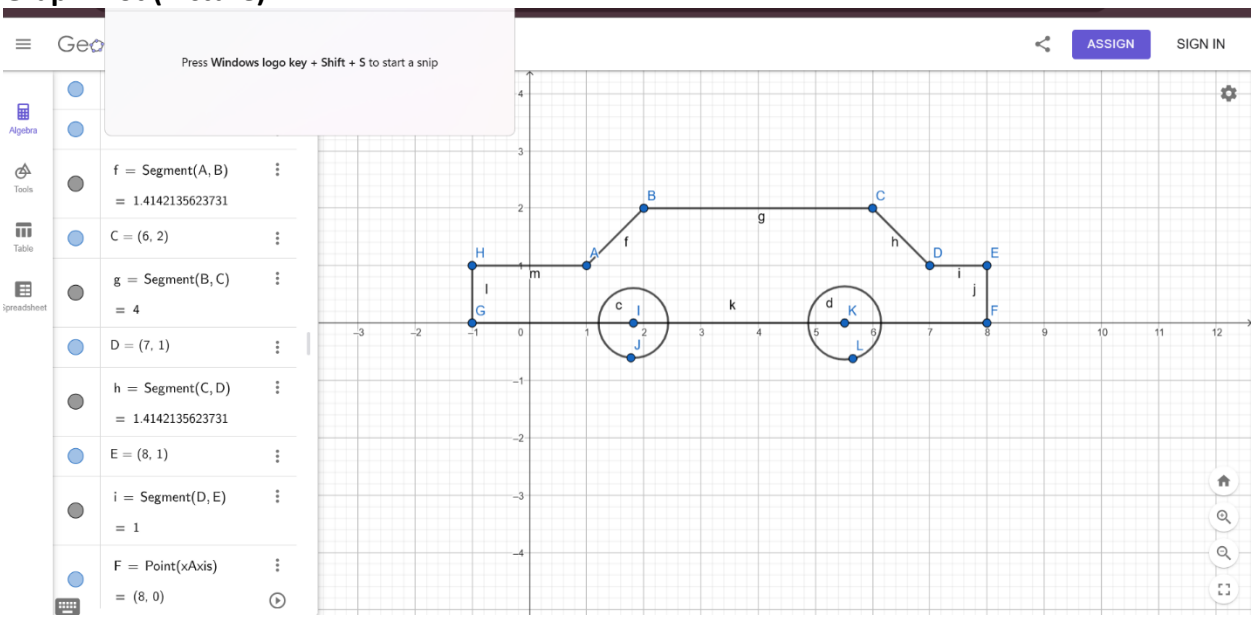
Output Screenshot (Full Screen)-



Question-2

Design a car which will have rotating wheels.

Graph Plot (Picture)-



Code-

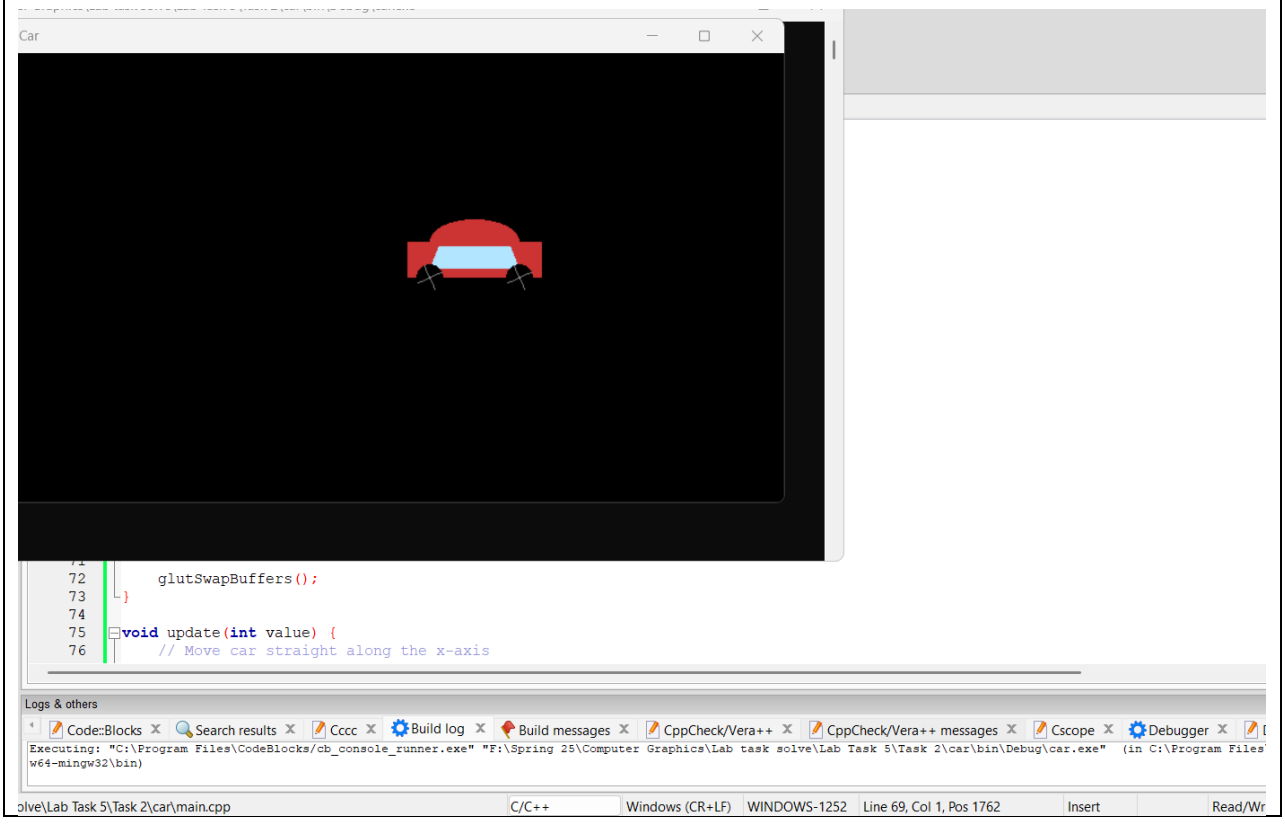
```
#include <GL/glut.h>
#include <cmath>
```

```
float carX = -8.0f; // Start from left
float carSpeed = 0.05f;
float wheelAngle = 0.0f;

void drawWheel(float x, float y) {
   glColor3f(0.0f, 0.0f, 0.0f);
   glBegin(GL_POLYGON);
   for(int i = 0; i < 360; i++) {
      float angle = i * 3.14159f / 180.0f;
      glVertex2f(x + 0.3f * cos(angle), y + 0.3f * sin(angle));
   }
   glEnd();

   glColor3f(0.5f, 0.5f, 0.5f);
   glPushMatrix();
   glTranslatef(x, y, 0.0f);
   glRotatef(wheelAngle, 0.0f, 0.0f, 1.0f);
   for(int i = 0; i < 4; i++) {
      glBegin(GL_LINES);
      glVertex2f(0.0f, 0.0f);
      glVertex2f(0.3f * cos(i * 90 * 3.14159f / 180.0f),
            0.3f * sin(i * 90 * 3.14159f / 180.0f));
      glEnd();
   }
   glPopMatrix();
}

void drawCar() {
   glColor3f(0.8f, 0.2f, 0.2f); // Red color
   glBegin(GL_POLYGON);
      glVertex2f(carX - 1.5f, 0.0f);
      glVertex2f(carX + 1.5f, 0.0f);
      glVertex2f(carX + 1.5f, 0.8f);
      glVertex2f(carX - 1.5f, 0.8f);
   glEnd();

   glBegin(GL_POLYGON);
   for(int i = 0; i < 180; i++) {
      float angle = i * 3.14159f / 180.0f;
      glVertex2f(carX + 1.0f * cos(angle), 0.8f + 0.5f * sin(angle));
   }
   glEnd();
```

```
    // Windows
    glColor3f(0.7f, 0.9f, 1.0f);
    glBegin(GL_POLYGON);
        glVertex2f(carX - 1.0f, 0.2f);
        glVertex2f(carX + 1.0f, 0.2f);
        glVertex2f(carX + 0.8f, 0.7f);
        glVertex2f(carX - 0.8f, 0.7f);
    glEnd();

    // Wheels
    drawWheel(carX - 1.0f, 0.0f);
    drawWheel(carX + 1.0f, 0.0f);
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);


    drawCar();

    glutSwapBuffers();
}

void update(int value) {
    // Move car straight along the x-axis
    carX += carSpeed;
    if(carX > 10.0f) {
        carX = -10.0f; // Reset car to the left when it moves off the right side
    }

    // Rotate wheels forward
    wheelAngle += carSpeed * 50;
    if(wheelAngle > 360.0f) wheelAngle -= 360.0f;

    glutPostRedisplay();
    glutTimerFunc(16, update, 0);
}

void init() {
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f); // Set background color to black
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-10.0, 10.0, -5.0, 5.0);
}
```

```
int main(int argc, char** argv) {
   glutInit(&argc, argv);
   glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
   glutInitWindowSize(800, 400);
   glutCreateWindow("Straight Moving Car");

   init();
   glutDisplayFunc(display);
   glutTimerFunc(0, update, 0);
   glutMainLoop();

   return 0;
}
```

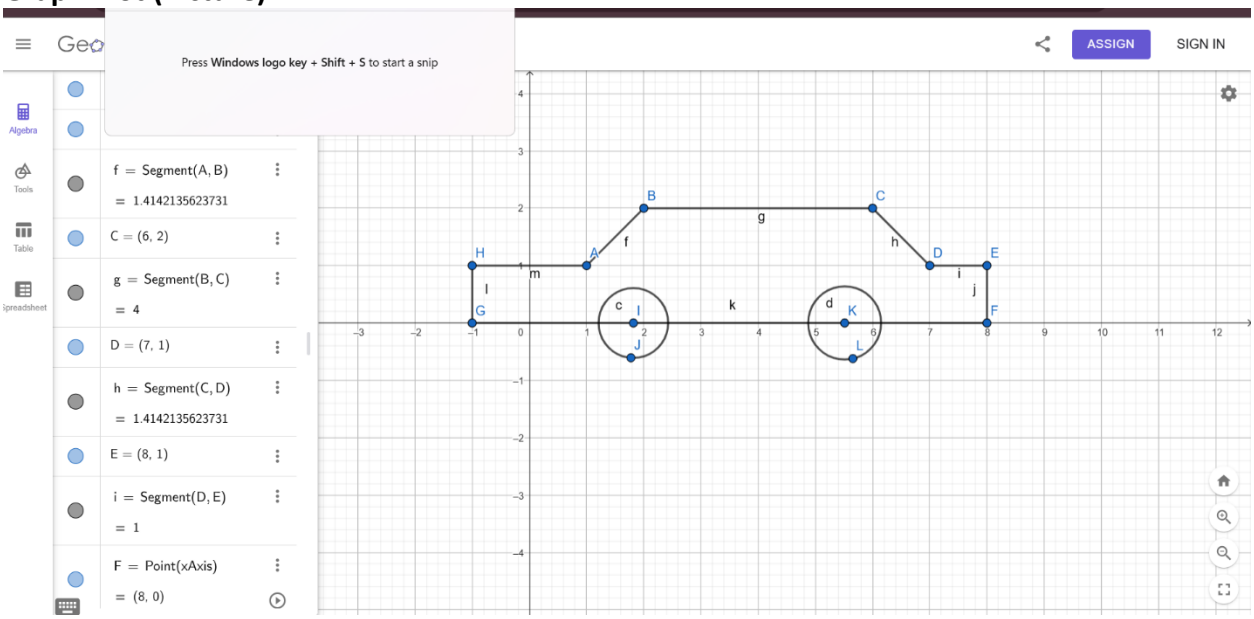
Output Screenshot (Full Screen)-



Question-3

Now move your car of question-2 from left to right in a loop.

Graph Plot (Picture)-



Code-

```
#include <GL/glut.h>
#include <cmath>

// Car properties
float carX = -8.0f; // Start from left
float carSpeed = 0.05f;
float wheelAngle = 0.0f;
bool moveRight = true;

void drawWheel(float x, float y) {
   // Wheel rim
   glColor3f(0.0f, 0.0f, 0.0f);
   glBegin(GL_POLYGON);
   for(int i = 0; i < 360; i++) {
      float angle = i * 3.14159f / 180.0f;
      glVertex2f(x + 0.3f * cos(angle), y + 0.3f * sin(angle));
   }
   glEnd();

   // Rotating spokes
```

```
    glColor3f(0.5f, 0.5f, 0.5f);
    glPushMatrix();
    glTranslatef(x, y, 0.0f);
    glRotatef(wheelAngle, 0.0f, 0.0f, 1.0f);
    for(int i = 0; i < 4; i++) {
        glBegin(GL_LINES);
        glVertex2f(0.0f, 0.0f);
        glVertex2f(0.3f * cos(i * 90 * 3.14159f / 180.0f),
                0.3f * sin(i * 90 * 3.14159f / 180.0f));
        glEnd();
    }
    glPopMatrix();
}

void drawCar() {
    // Car body
    glColor3f(0.8f, 0.2f, 0.2f); // Red color
    glBegin(GL_POLYGON);
        glVertex2f(carX - 1.5f, 0.0f);
        glVertex2f(carX + 1.5f, 0.0f);
        glVertex2f(carX + 1.5f, 0.8f);
        glVertex2f(carX - 1.5f, 0.8f);
    glEnd();

    // Car top
    glBegin(GL_POLYGON);
    for(int i = 0; i < 180; i++) {
        float angle = i * 3.14159f / 180.0f;
        glVertex2f(carX + 1.0f * cos(angle), 0.8f + 0.5f * sin(angle));
    }
    glEnd();

    // Windows
    glColor3f(0.7f, 0.9f, 1.0f);
    glBegin(GL_POLYGON);
        glVertex2f(carX - 1.0f, 0.2f);
        glVertex2f(carX + 1.0f, 0.2f);
        glVertex2f(carX + 0.8f, 0.7f);
        glVertex2f(carX - 0.8f, 0.7f);
    glEnd();

    // Wheels
    drawWheel(carX - 1.0f, 0.0f);
    drawWheel(carX + 1.0f, 0.0f);
```

```
}

void display() {
   glClear(GL_COLOR_BUFFER_BIT);

   // Road background
   glColor3f(0.3f, 0.3f, 0.3f); // Gray road
   glBegin(GL_POLYGON);
      glVertex2f(-10.0f, -2.0f);
      glVertex2f(10.0f, -2.0f);
      glVertex2f(10.0f, 0.0f);
      glVertex2f(-10.0f, 0.0f);
   glEnd();

   // Grass background
   glColor3f(0.0f, 0.5f, 0.0f);
   glBegin(GL_POLYGON);
      glVertex2f(-10.0f, -5.0f);
      glVertex2f(10.0f, -5.0f);
      glVertex2f(10.0f, -2.0f);
      glVertex2f(-10.0f, -2.0f);
   glEnd();

   // Sky background
   glColor3f(0.6f, 0.8f, 1.0f);
   glBegin(GL_POLYGON);
      glVertex2f(-10.0f, 0.0f);
      glVertex2f(10.0f, 0.0f);
      glVertex2f(10.0f, 5.0f);
      glVertex2f(-10.0f, 5.0f);
   glEnd();

   drawCar();

   glutSwapBuffers();
}

void update(int value) {
   // Move car straight only
   if(moveRight) {
      carX += carSpeed;
      if(carX > 10.0f) {
         carX = -10.0f; // Reset to left when exits right
      }
```

```
    }

    // Rotate wheels forward only
    wheelAngle += carSpeed * 50;
    if(wheelAngle > 360.0f) wheelAngle -= 360.0f;

    glutPostRedisplay();
    glutTimerFunc(16, update, 0);
}

void init() {
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-10.0, 10.0, -5.0, 5.0);
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(800, 400);
    glutCreateWindow("Straight Moving Car");

    init();
    glutDisplayFunc(display);
    glutTimerFunc(0, update, 0);
    glutMainLoop();

    return 0;
}
```

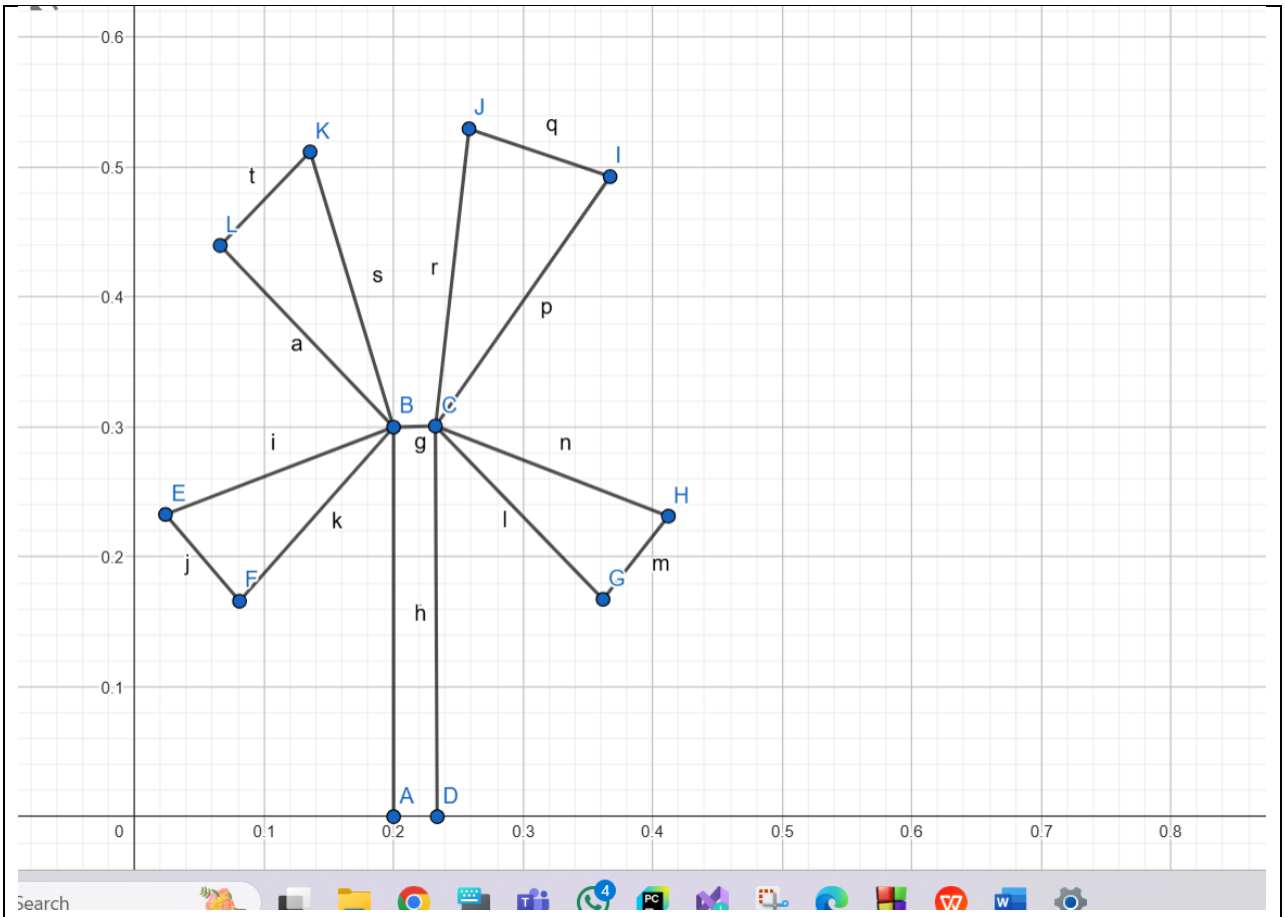
Output Screenshot (Full Screen)-

```
    wheelAngle += carSpeed * 50;
    if(wheelAngle > 360.0f) wheelAngle -= 360.0f;

    glutPostRedisplay();
    glutTimerFunc(16, update, 0);
```

Question-4

Design a windmill with rotating blades

Graph Plot (Picture)-



Code-

```
#include <GL/glut.h>
#include <cmath>

// Windmill parameters
float bladeAngle = 0.0f;
float rotationSpeed = 2.0f; // Faster rotation
const int NUM_BLADES = 4;   // 4 blades (pakha)

void init() {
   glClearColor(1.0f, 1.0f, 1.0f, 1.0f); // White background
   glMatrixMode(GL_PROJECTION);
   gluOrtho2D(-1.0, 1.0, -1.0, 1.0);
}

void drawWindmill() {
   // Draw tower (brown rectangle)
   glColor3f(0.55f, 0.27f, 0.07f); // Brown
```

```
    glBegin(GL_QUADS);
        glVertex2f(-0.03f, -1.0f);
        glVertex2f(0.03f, -1.0f);
        glVertex2f(0.03f, 0.0f);
        glVertex2f(-0.03f, 0.0f);
    glEnd();

    // Draw hub (center circle)
    glColor3f(0.3f, 0.3f, 0.3f); // Dark gray
    glBegin(GL_POLYGON);
        for(int i = 0; i < 360; i++) {
            float degInRad = i * M_PI/180.0f;
            glVertex2f(cos(degInRad)*0.08, sin(degInRad)*0.08 + 0.1f);
        }
    glEnd();

    // Draw 4 rotating blades (pakha)
    glColor3f(0.5f, 0.5f, 0.8f); // Light blue blades
    for(int i = 0; i < NUM_BLADES; i++) {
        float angle = bladeAngle + i * (360.0f/NUM_BLADES);
        float rad = angle * M_PI / 180.0f;

        // Each blade is a quadrilateral
        glBegin(GL_QUADS);
            // Base near hub
            glVertex2f(cos(rad)*0.08, sin(rad)*0.08 + 0.1f);
            glVertex2f(cos(rad + 7)*0.08, sin(rad + 7)*0.08 + 0.1f);
            // Tip of blade
            glVertex2f(cos(rad + 7)*0.7, sin(rad + 7)*0.7 + 0.1f);
            glVertex2f(cos(rad)*0.7, sin(rad)*0.7 + 0.1f);
        glEnd();
    }
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    drawWindmill();

    glutSwapBuffers();
}

void update(int value) {
    bladeAngle += rotationSpeed;
```

```
    if(bladeAngle > 360) bladeAngle -= 360;

    glutPostRedisplay();
    glutTimerFunc(16, update, 0); // ~60 FPS
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);
    glutInitWindowSize(600, 600);
    glutCreateWindow("4-Blade Windmill (Pakha)");

    init();
    glutDisplayFunc(display);
    glutTimerFunc(25, update, 0);

    glutMainLoop();
    return 0;
}
```

Output Screenshot (Full Screen)-